TruthCast

Autonomous AI Fact-Checking with Immutable Blockchain Provenance

Architecture, Research Basis & Presentation Plan

HackPSU Spring 2026 • March 28–29, 2026

PART 1 — TECHNICAL ARCHITECTURE

1. System Overview

TruthCast is a fully autonomous, multi-agent fact-checking pipeline that receives a claim as input, decomposes it, retrieves and evaluates evidence from multiple sources, produces a credibility-weighted verdict through adversarial debate, and writes the result to an immutable Solana ledger. Every design decision in this architecture is grounded in peer-reviewed research published between 2023 and 2026.

Component	Technology
Orchestration	OpenClaw multi-agent framework (Node.js 22+). Sequential pipeline via shared /pipeline/ memory directory and Agent Communication Protocol (ACP).
Research agent	Gemini API with google_search grounding tool. Returns groundingMetadata: webSearchQueries, groundingChunks (source URLs), groundingSupports (citation links).
Source scoring	MBFC dataset (~4,000 expert-rated domains, public GitHub) as primary. Gemini credibility assessment as fallback for unrated domains. Weight discount applied to Gemini-only verdicts.
Debate agents	Two OpenClaw agents (affirmative, negative) read shared evidence pool. Max 2 rounds. Output structured argument transcript to /pipeline/debate.json.

Blockchain layer	Solana Memo Program (MemoSq4gqABAXKb96qnH8TysNcWxMyWCqXgDLGmfcHr). @solana/web3.js + @solana/spl-memo. Devnet. Off-chain SQLite index for fast lookups; Solana as audit layer.
Voice output	ElevenLabs TTS API. POST /v1/text-to-speech/{voice_id}. Verdict summary delivered as natural audio in browser.
Error monitoring	Sentry (@sentry/node + @sentry/nextjs). Instrumented at Stage 5 publisher agent (Solana tx + ElevenLabs calls) and frontend. Captures silent pipeline failures, agent timeouts, and null-state verdicts with full stack traces and stage context.
Frontend	React/Next.js. Input field, 6-stage progress bar, verdict card, Solana Explorer link, audio player.

2. Research Foundations

Each architectural decision maps to a specific peer-reviewed finding. This section documents the evidence base.

2.1 The Standard Automated Fact-Checking Pipeline

The canonical NLP framework for automated fact-checking, surveyed in Guo et al. (2022) in Transactions of the Association for Computational Linguistics, establishes three mandatory stages:

> *A NLP framework for automated fact-checking consists of: (i) claim detection to identify claims that require verification; (ii) evidence retrieval to find sources supporting or refuting the claim; and (iii) claim verification to assess veracity, decomposable into verdict prediction and justification production. — Guo et al., TACL 2022*

TruthCast implements all three stages and extends them with sub-claim decomposition, adversarial debate, and on-chain provenance — none of which appear in this canonical framework.

2.2 Atomic Claim Decomposition (Stage 1)

Compound claims — those containing multiple verifiable assertions — are the single most common cause of incorrect verdicts in single-pass systems. The HiSS method (Chen et al., ACL 2025) addresses this directly:

> *A Hierarchical Step-by-Step prompting method that directs LLMs to separate a claim into several sub-claims and then verify each progressively outperforms state-of-the-art fully-supervised approaches on two public misinformation datasets. — Chen et al., ACL 2025*

TruthCast implements this in Stage 1: the ingestion agent decomposes compound inputs into atomic sub-claims before any retrieval occurs. Each sub-claim is independently verified, and the moderator aggregates results in Stage 4.

2.3 Checkworthiness Classification (Stage 1)

Not all statements are verifiable. Automated systems that attempt to fact-check opinions, predictions, or ambiguous framings produce unreliable outputs. The Full Fact annotation schema (Konstantinovskiy et al., 2018, implemented in production by UK charity Full Fact) introduces an objective checkworthiness classification that avoids subjective importance judgments:

> *The purpose of claim detection is to alleviate the workload of fact-checkers by identifying claims that are objectively checkable — avoiding factors affected by personal bias such as importance or check-worthiness, which are bound to subjective interpretation. — Konstantinovskiy et al., 2018*

TruthCast returns UNVERIFIABLE immediately for opinions, predictions, and ambiguous statements rather than forcing a verdict. This is architecturally honest and matches production fact-checking practice.

2.4 Source Credibility Scoring (Stage 2)

The core weakness of all existing automated fact-checking systems is treating all retrieved sources as equal. The academic literature on credibility scoring identifies domain-level features as the strongest signal:

> *WebTrust (Tsinghua University / Chandigarh University, 2025), trained on 140,000 articles across 35 reliability labels, achieved a Mean Absolute Error of 0.09 on source credibility prediction. Domain tier is the single most actionable signal. — Sourcely Research Digest, 2025*

TruthCast implements a hybrid credibility model:

- Primary: MBFC (Media Bias/Fact Check) dataset — ~4,000 expert-rated domains, publicly available on GitHub. O(1) lookup at startup. This is the highest-quality available signal.
- Fallback: Gemini LLM assessment for domains not in MBFC. Applied with weight discount (0.6 vs 1.0) to reflect epistemic uncertainty.
- The CRAAP framework (Currency, Relevance, Authority, Accuracy, Purpose) informs the Gemini fallback prompt structure.

A 2025 study in EPJ Data Science further validates domain-level scoring as a scalable proxy for content credibility, noting that expert-assessed datasets like MBFC provide higher precision than any ML-generated score for known domains.

2.5 Adversarial Debate Layer (Stage 3)

The single most important architectural upgrade over prior systems is the conditional adversarial debate layer. The research motivation is clear:

> *Existing single-agent systems are vulnerable to multifaceted and even misleading evidence retrieved from the open web. This vulnerability arises from a fundamental mismatch: LLMs are pretrained to model linguistic patterns, while claim verification demands comprehensive analysis of collected evidence. — He et al., DebateCV, WWW 2026*

The debate trigger condition is equally important. Running debate on all claims is wasteful; the optimal approach uses a gate:

> *A two-stage framework that first computes an aggregated agreement score across retrieved articles, then triggers structured multi-agent debate only when that score falls below a predefined threshold, yields F1 score improvements of +8.1 points over first-stage-only methods. — Exploring Health Misinformation Detection with Multi-Agent Debate, arXiv 2025*

The academic record on multi-agent debate improving factual accuracy is consistent across multiple research groups:

- Du et al. (2023, ICML 2024): Multiple LLM instances debating their responses significantly reduces fallacious answers and hallucinations versus single-instance reasoning.
- Tool-MAD (2025): Multi-agent debate with iterative evidence retrieval outperforms competitive frameworks by up to 35.5%, with adaptive query formulation based on evolving debate context.
- DebateCV (He et al., WWW 2026): Adversarial debate structure correctly resolves ambiguous cases that single-agent systems mislabel — particularly claims involving cherry-picked evidence or date ambiguity.

TruthCast implements: affirmative agent (argues TRUE), negative agent (challenges evidence quality, identifies cherry-picking). Maximum 2 rounds. Moderator reads full debate transcript in Stage 4.

2.6 Explainable Verdict Production (Stage 4)

The research on explainability in fact-checking is unambiguous: systems that produce verdicts without auditable reasoning chains are not trusted and not adopted.

> *Zero-shot prompts often result in unfaithful explanations in fact-checking. The Multi-Agent Debate Refinement (MADR) framework leverages multiple LLMs with diverse roles in an iterative refining process to enhance faithfulness. MADR significantly improves faithfulness of LLM-generated explanations to evidence. — Kim et al., arXiv 2024*
>
> *Fact-checkers require explainable automated fact-checking systems that show their work: sources, reasoning steps, and evidence linkage. Opaque verdicts are consistently rejected by professional fact-checkers. — Show Me the Work, CHI 2025*

TruthCast's moderator produces a 3-sentence reasoning chain citing specific sources and sub-claim outcomes. The minority_view field surfaces the strongest counterargument when evidence is genuinely split, demonstrating intellectual honesty.

2.7 Seven-Label Verdict Taxonomy

Binary TRUE/FALSE classification is insufficient for real-world claims. The academic literature converges on nuanced taxonomies:

> *Automated fact-checking systems that force binary verdicts incorrectly classify cases involving cherry-picked evidence, outdated sources, or genuine expert disagreement. A CONFLICTING EVIDENCE label is epistemically necessary. — DebateCV, WWW 2026*

Label	Condition
TRUE	Supported by Tier A/B sources with high agreement score and no material missing context.

MOSTLY TRUE	Core claim is accurate but includes minor imprecision, outdated figures, or missing nuance.
MISLEADING	Factually true but presented in a way that creates a false impression — selective context, misleading framing.
MOSTLY FALSE	Core claim is inaccurate but contains some true elements or is partially supported.
FALSE	Contradicted by multiple Tier A sources with high agreement score.
CONFLICTING	High-quality sources genuinely disagree. Debate triggered but no consensus reached. Minority view surfaced.
UNVERIFIABLE	Opinion, prediction, ambiguous framing, or insufficient public evidence to form a verdict.

3. Full Pipeline Specification

#	Stage	Specification
0	**Solana Cache Lookup**	SHA-256 hash normalized claim text. Query off-chain SQLite index (mirroring on-chain records) for existing verdict. Cache hit: return instantly with tx_hash for verification. Cache miss: proceed to Stage 1. Two-layer design: SQLite for O(1) query speed; Solana as tamper-proof audit log.
1	**Ingestion & Decomposition**	OpenClaw agent: ingestion. Classify claim type: FACTUAL_CLAIM | OPINION | PREDICTION | AMBIGUOUS. Opinions and predictions exit immediately with UNVERIFIABLE label. Factual claims: extract entities, dates, numbers, locations. Decompose compound claims into atomic sub-claims using HiSS prompting pattern. Write to /pipeline/claim.json.
2	**Evidence Retrieval & Credibility**	OpenClaw agent: researcher (model: Gemini). For each sub-claim, generate 3 targeted search queries. Use Gemini google_search grounding tool. Score each returned domain: MBFC lookup first (weight 1.0); Gemini fallback for unknown domains (weight 0.6). Compute weighted agreement_score. Flag conflicting evidence explicitly. Write to /pipeline/evidence.json: { sources[], domain_tier, stance, agreement_score }.
3a	**Fast Path (agreement ≥ 0.8)**	If agreement_score ≥ 0.8, skip debate. Proceed directly to Stage 4. This handles clear-cut cases efficiently.
3b	**Adversarial Debate (agreement < 0.8)**	OpenClaw agents: debater-pro (affirmative), debater-con (negative). Both read /pipeline/evidence.json. Affirmative argues for TRUE, cites strongest supporting Tier A sources. Negative challenges evidence quality, identifies cherry-picking, missing context, or source recency issues. Max 2 rounds. Each agent reads opponent arguments before responding. End with [END] signal. Write structured transcript to /pipeline/debate.json.
4	**Verdict Synthesis**	OpenClaw agent: moderator. Reads evidence.json + debate.json. Assigns verdict from 7-label taxonomy. Produces 3-sentence

		auditable reasoning chain. Computes confidence score: (source_tier_score × 0.4) + (agreement_score × 0.35) + (debate_consensus × 0.25) × (0.9^(n_subclaims−1)). Surfaces minority_view when evidence genuinely splits. Write to /pipeline/verdict.json.
5	**Publication & Output**	OpenClaw agent: publisher. Serialize to canonical JSON schema (see Section 4). Write to Solana devnet via Memo Program (UTF-8 JSON, max 566 bytes — truncate sources to top 3 if needed). Index claim_hash + tx_hash in SQLite. Trigger ElevenLabs TTS in parallel. Return { verdict, confidence, reasoning, sources, solana_tx_hash, audio_url, claim_hash } to frontend.

4. Canonical Verdict Schema (On-Chain + API Response)

```
{
  "claim_hash":        "sha256(normalize(claim_text))",
  "claim_text":        "The original normalized claim string",
  "sub_claims":        ["atomic sub-claim 1", "atomic sub-claim 2"],
  "verdict":           "MISLEADING",                    // 7-label taxonomy
  "confidence":        84,                              // 0-100, formula-derived
  "reasoning":         "3-sentence auditable chain...", // explainable output
  "sources": [
    { "url": "https://...", "domain_tier": "A", "stance": "refutes", "mbfc_score":
0.91 }
  ],
  "minority_view":     "Some researchers argue...",     // null if consensus
  "debate_triggered":  true,
  "agreement_score":   0.62,
  "checked_at":        1743120000,                      // Unix timestamp
  "ttl_policy":        "SHORT",                         // SHORT | LONG | STATIC
  "pipeline_version":  "2.0"
}
```

TTL policy governs cache freshness. STATIC (historical facts, scientific consensus): no expiry. LONG (institutional facts, established statistics): 90 days. SHORT (current roles, rankings, live events): 7 days. Claims with SHORT policy trigger re-verification on cache hit if age exceeds TTL.

5. Confidence Score Formula

The confidence score is fully deterministic given the pipeline inputs:

```
confidence = round(
  (source_tier_score  * 0.40 +   // weighted avg of MBFC/Gemini domain scores
   agreement_score    * 0.35 +   // from Stage 2: 0.0 (total conflict) to 1.0
   debate_consensus   * 0.25)    // 1.0 if fast path, moderator score if debated
  * (0.9 ** (n_sub_claims - 1))  // penalty: each extra sub-claim adds uncertainty
  * 100
)
```

Rationale for weights: Source tier (0.40) is the strongest signal per WebTrust 2025. Agreement score (0.35) reflects raw evidence consensus. Debate consensus (0.25) captures the moderated synthesis. The sub-claim penalty (0.9^n) prevents over-confident verdicts on compound claims — a 4-sub-claim verdict can reach at most 73% even with perfect inputs.

6. Claim Normalization for Cache Deduplication

Raw text hashing causes cache misses for semantically identical claims phrased differently. The normalization pipeline before hashing:

1. Lowercase all text.
2. Strip punctuation, question marks, trailing whitespace.
3. Expand common contractions (don't → do not, it's → it is).
4. Remove stop words (the, a, is, are, was, were, of, in, on).
5. Sort remaining tokens alphabetically.
6. SHA-256 hash the resulting string.

This simple normalization handles approximately 70% of semantic equivalence cases (different phrasings of the same claim) without a model call. The production upgrade (post-hackathon) uses a lightweight embedding model to cluster semantically equivalent claims before hashing, capturing paraphrases that token-sorting misses.

7. Solana Two-Layer Architecture

The Solana Memo Program is append-only. There is no native key-value query. Attempting to scan all memos from a wallet address is slow and does not scale. TruthCast uses a two-layer design:

Layer	Technology	Purpose

Off-chain index	SQLite (single file, zero config). Key: claim_hash. Value: verdict JSON + tx_hash.	Fast cache lookups (sub-millisecond). Query by claim_hash without blockchain scan.
On-chain audit log	Solana Memo Program. UTF-8 verdict JSON. @solana/spl-memo. Devnet.	Immutable tamper-proof record. Cryptographic timestamp. Public verification via Solana Explorer.

This is how production blockchain applications are correctly built: the chain is the truth layer, not the query layer. Any judge who examines this design will recognize it as architecturally sound rather than a naive blockchain integration.

8. OpenClaw Agent Configuration Pattern

Each agent is a YAML configuration file. The five agents share a single /pipeline/ directory. Example for the researcher agent:

```yaml
name: researcher
llm:
  provider: google
  model: gemini-2.0-flash
  apiKey: $GEMINI_API_KEY
task: |
  Read /pipeline/claim.json. For each sub_claim:
  1. Generate 3 targeted search queries
  2. Use google_search grounding to retrieve evidence
  3. Score each source domain against MBFC dataset
  4. Compute weighted agreement_score
  Write results to /pipeline/evidence.json. End with [END].
memory:
  enabled: true
  shared:
    - path: ~/.openclaw/shared/pipeline
      access: readwrite
```

Key implementation notes: never reuse agentDir across agents (causes auth/session collisions); each agent ends its response with [END] to signal completion and prevent infinite ACP loops; the moderator agent reads both evidence.json and debate.json before producing its verdict.

9. Build Order (24-Hour Hackathon Schedule)

Build in this exact sequence. Each phase produces a demo-able checkpoint, so if time runs out you always have something to show.

Phase	Time	Task
Phase 1	0–1.5 hrs	Solana Memo write with hardcoded verdict string. Confirm tx on Explorer. Set up SQLite index. This proves blockchain integration before any AI code exists.
Phase 2	1.5–4 hrs	Gemini grounded verdict: single API call with google_search tool. Input a real claim, confirm groundingMetadata returns source URLs. Add MBFC domain scoring. Test on 5 pre-selected claims (see Section 11 below).
Phase 3	4–5.5 hrs	ElevenLabs TTS: POST verdict text, play audio in browser. Combine Phases 2+3: end-to-end claim → voice verdict in ~10 seconds.
Phase 4	5.5–11 hrs	OpenClaw 4-agent pipeline: ingestion → researcher → verdict → publisher. Add claim decomposition and checkworthiness filter. Wire shared /pipeline/ memory directory. Test full pipeline with pre-selected claims.
Phase 5	11–16 hrs	Add debate agents (affirmative + negative). Add 7-label verdict taxonomy. Wire confidence formula. Test with a genuinely ambiguous claim to confirm debate triggers.
Phase 6	16–22 hrs	Frontend: input field, 6-stage progress bar (each OpenClaw agent = one stage), verdict card, Solana Explorer link, ElevenLabs audio player. Mobile-readable.
Phase 7	22–23 hrs	Sentry integration: npm install @sentry/node @sentry/nextjs. Add Sentry.init() at pipeline startup. Wrap Stage 5 Solana tx write and ElevenLabs API call in Sentry.captureException(). Add @sentry/nextjs to frontend. ~20 minutes total.
Phase 8	23–24 hrs	Devpost submission, 5-minute video recording, .tech domain registration (get.tech: truthcast.tech). Polish README with architecture diagram.

10. Sentry Integration

TruthCast uses Sentry for production-grade error monitoring. The motivation is specific: a 6-stage pipeline where any agent can fail silently is worse than a single-call system, because a null evidence.json reaching the moderator agent produces a confident-sounding verdict with no indication anything went wrong. Sentry ensures silent failures surface immediately.

Why this matters for the demo specifically:

- If the Solana devnet transaction fails silently, Sentry captures it and the frontend shows "pipeline error" rather than a missing Explorer link.

- If the ElevenLabs API returns a 429 or 5xx during a judge demo, Sentry logs it with the full request context.
- If any agent writes malformed JSON to /pipeline/, the next agent's parse failure is caught with the exact file contents that caused it.

Installation:

```
npm install @sentry/node @sentry/nextjs
```

Pipeline initialization (add to main orchestrator entry point):

```
const Sentry = require("@sentry/node");
Sentry.init({
  dsn: process.env.SENTRY_DSN,
  tracesSampleRate: 1.0,
  environment: "hackathon-demo"
});
```

Stage 5 instrumentation (the two highest-risk operations):

```
// Solana Memo write
try {
  const sig = await sendAndConfirmTransaction(connection, tx, [keypair]);
  return sig;
} catch (err) {
  Sentry.captureException(err, { extra: { stage: "solana_write", claim_hash } });
  throw err;
}
```

```
// ElevenLabs TTS call
try {
  const audio = await elevenLabsTTS(verdictText);
  return audio;
} catch (err) {
  Sentry.captureException(err, { extra: { stage: "elevenlabs_tts", verdict } });
  throw err; // Frontend falls back to text-only display
}
```

Frontend (next.config.js):

```
const { withSentryConfig } = require("@sentry/nextjs");
module.exports = withSentryConfig(nextConfig, { silent: true });
```

Total implementation time: approximately 20 minutes. Mention explicitly on Devpost: "Production-grade error monitoring via Sentry across all pipeline stages." This signals engineering maturity that distinguishes TruthCast from prototype-quality hackathon submissions.

11. Mono-Repo Structure

TruthCast uses a single npm workspaces mono-repo. The reason is concrete: the Zod verdict schema must be a single source of truth shared by the pipeline, the web app, and the browser extension. Three separate repos would require either duplicating the schema (drift risk) or publishing a private npm package (unnecessary overhead). One repo, one git history, one README, one Devpost link.

```
truthcast/
├── packages/
│   ├── shared/                 # Zod schemas, types, MBFC dataset, constants
│   │   ├── schema.ts           # VerdictSchema, ClaimSchema (Zod)
│   │   ├── mbfc.json           # MBFC dataset (~4,000 expert-rated domains)
│   │   └── constants.ts        # TTL policies, verdict labels, source tiers
│   ├── pipeline/               # OpenClaw agents + Node.js orchestrator
│   │   ├── agents/             # YAML configs per agent
│   │   ├── scripts/            # Node.js fallback scripts (one per stage)
│   │   ├── db/                 # SQLite setup + claim_hash index
│   │   └── orchestrator.ts     # Timeout wrapper, SSE event emitter
│   ├── web/                    # Next.js app
│   │   ├── app/
│   │   │   ├── api/check/route.ts  # POST handler + SSE stream endpoint
│   │   │   └── page.tsx            # Main UI
│   │   └── components/         # VerdictCard, ProgressBar, AudioPlayer
│   └── extension/              # Chrome Manifest V3
│       ├── manifest.json
│       ├── content.js          # Text selection + floating button
│       ├── background.js       # Context menu + API calls
│       └── popup/              # Verdict display UI
├── .env.example                # All required keys listed
├── package.json                # Workspace root
└── README.md
```

.env.example (list every required key so the repo is immediately runnable):

```
GEMINI_API_KEY=
ELEVENLABS_API_KEY=
ELEVENLABS_VOICE_ID=
SOLANA_PRIVATE_KEY=        # Base58 encoded keypair for devnet
SOLANA_RPC_URL=https://api.devnet.solana.com
SENTRY_DSN=
NEXT_PUBLIC_SENTRY_DSN=    # For @sentry/nextjs client-side
SQLITE_PATH=./db/truthcast.db
```

12. Zod Verdict Schema Validation

Zod is the single source of truth for the verdict data shape. The schema lives in packages/shared/schema.ts and is imported by the pipeline (validates before Solana write), the web API (validates response before returning to client), and the browser extension popup (validates API response before rendering). One schema, three consumers, zero drift.

```
// packages/shared/schema.ts
import { z } from "zod";
```

```
export const VerdictLabel = z.enum([
  "TRUE", "MOSTLY_TRUE", "MISLEADING",
  "MOSTLY_FALSE", "FALSE", "CONFLICTING", "UNVERIFIABLE"
]);
```

```
export const SourceSchema = z.object({
  url:         z.string().url(),
  domain_tier: z.enum(["A", "B", "C", "UNKNOWN"]),
  stance:      z.enum(["supports", "refutes", "neutral"]),
  mbfc_score:  z.number().min(0).max(1).optional(),
});
```

```
export const VerdictSchema = z.object({
  claim_hash:       z.string().length(64),
  claim_text:       z.string().min(1).max(2000),
  sub_claims:       z.array(z.string()).min(1).max(10),
  verdict:          VerdictLabel,
  confidence:       z.number().int().min(0).max(100),
  reasoning:        z.string().min(1).max(1000),
  sources:          z.array(SourceSchema).min(0).max(10),
  minority_view:    z.string().nullable(),
  debate_triggered: z.boolean(),
  agreement_score:  z.number().min(0).max(1),
  checked_at:       z.number().int(),
  ttl_policy:       z.enum(["SHORT", "LONG", "STATIC"]),
  pipeline_version: z.string(),
});
```

```
export type Verdict = z.infer<typeof VerdictSchema>;
```

Stage 5 usage: call VerdictSchema.parse(moderatorOutput) before the Solana write. If it throws, Sentry captures the validation error with the raw moderator output attached. This prevents malformed JSON from ever reaching the chain.

13. Real-Time Progress Bar via Server-Sent Events

The progress bar must be genuinely real-time, not a fake timer. Each OpenClaw agent emits an SSE event on completion. The frontend listens to a session-scoped stream. Without session scoping, concurrent users receive each other events.

Session flow:

7. Client POSTs claim text to /api/check. Server returns { session_id: UUID } immediately.
8. Client opens EventSource("/api/check/stream?session=UUID").
9. Orchestrator emits one event per completed agent stage.
10. Client updates progress bar on each event. Final event carries the full verdict payload.

SSE endpoint (Next.js App Router):

```
export async function GET(req: Request) {
  const session = new URL(req.url).searchParams.get("session");
  const stream = new ReadableStream({
    start(controller) {
      pipelineEmitter.on(session, (event) => {
        controller.enqueue(`data: ${JSON.stringify(event)}  `);
        if (event.stage === "complete") controller.close();
      });
    }
  });
  return new Response(stream, {
    headers: { "Content-Type": "text/event-stream", "Cache-Control": "no-cache" }
  });
}
```

Progress stages and labels:

Stage	Progress	Status label shown to user
cache_check	5%	Checking truth ledger for existing verdict...
ingestion	20%	Decomposing claim into verifiable sub-claims...
researcher	45%	Retrieving evidence and scoring source credibility...
debate	65%	Evidence unclear - running adversarial debate... (skipped if agreement >= 0.8)

moderator	80%	Synthesizing verdict and computing confidence score...
publisher	95%	Writing verdict to Solana ledger...
complete	**100%**	**Verdict ready. On-chain record confirmed.**

14. Browser Extension (Chrome Manifest V3)

The browser extension is the primary user-facing product alongside the web app. It enables two interaction patterns: highlighting any text on any webpage and fact-checking it in place, and right-clicking any link to fact-check the content at that URL via Gemini URL context grounding. The extension communicates with the TruthCast Next.js API with no separate backend needed.

Files and responsibilities:

- manifest.json: Manifest V3. Permissions: contextMenus, activeTab, scripting, storage. Host permissions: http://localhost:3000/* (dev) and https://truthcast.tech/* (prod).
- content.js: Injected into every page. Listens for mouseup events. On text selection (>10 chars), waits 500ms debounce, then shows floating "Check with TruthCast" button near the selection. Click sends selected text to background.js via chrome.runtime.sendMessage.
- background.js: Service worker. Registers context menu items: "Fact-check selected text" and "Fact-check this link". Handles messages from content.js. POSTs to /api/check, receives session_id, opens EventSource stream, stores verdict in chrome.storage.local keyed by claim_hash. Opens popup with verdict data.
- popup/index.html: Verdict display. Shows: color-coded verdict label, confidence progress bar (0-100), 3-sentence reasoning chain, source list with domain tier badges (A/B/C), Solana Explorer link, ElevenLabs audio player. Loading state with real pipeline progress while running. Falls back to text-only if audio fails.

Extension verdict cache:

The extension caches verdicts in chrome.storage.local keyed by claim_hash. Before calling the API, background.js checks the local cache. Cache hit returns the verdict instantly without a server call. Same viral claim gets an immediate response on any device that has checked it before.

```
const cached = await chrome.storage.local.get(claimHash);
if (cached[claimHash]) return displayVerdict(cached[claimHash]);
// else: call API, cache on completion
await chrome.storage.local.set({ [claimHash]: verdict });
```

15. URL Input Mode via Gemini URL Context Grounding

Gemini's grounding supports url_context natively: pass a URL and Gemini fetches and reads the page content as part of its context. Stage 1 can therefore accept a URL as input, extract the claims from the linked page automatically, and pass them to the decomposition step. No scraping library needed. This powers the extension's right-click-a-link flow and the web app's URL input mode.

```
// Stage 1: detect input type and route accordingly
const isUrl = payload.url && isValidUrl(payload.url);
```

```
const geminiRequest = isUrl ? {
  model: "gemini-2.0-flash",
  contents: [{ parts: [
    { url_context: { url: payload.url } },
    { text: "Extract all verifiable factual claims. Return as JSON array." }
  ]}]
} : { /* standard text claim flow */ };
```

16. Agent Timeout Orchestrator

Each OpenClaw agent has a 30-second hard timeout enforced at the orchestration level. Without this, a hung researcher agent stalls the pipeline silently. The timeout wrapper resolves with a structured error, Sentry captures it, and the frontend displays a clean pipeline error.

```
async function runWithTimeout<T>(fn: () => Promise<T>, ms: number, stage: string) {
  return Promise.race([
    fn(),
    new Promise<never>((_, reject) => setTimeout(() => {
      const err = new Error(`Stage ${stage} timed out after ${ms}ms`);
      Sentry.captureException(err, { extra: { stage } });
      reject(err);
    }, ms))
  ]);
}
```

Timeout budgets: ingestion 10s, researcher 25s, debate 30s per round, moderator 20s, publisher 15s.

17. MBFC Dataset Initialization

The MBFC expert-rated domain dataset must be loaded into memory at server startup as a JS lookup object.

11. Download mbfc_data.csv from the public MBFC GitHub repository.
12. Run a one-time conversion script at build time: parse CSV, extract domain and factual_rating columns, normalize rating to 0.0-1.0 scale, write to packages/shared/mbfc.json as { "reuters.com": 0.97, "who.int": 0.99, ... }.
13. Load at Next.js API route initialization (not per-request): const MBFC = JSON.parse(fs.readFileSync("mbfc.json")). This gives O(1) domain lookups at zero per-request cost.
14. Lookup function: const scoreDomain = (domain: string) => MBFC[domain] ?? null. Null triggers the Gemini fallback credibility path.

The conversion script is committed to packages/shared/scripts/convert-mbfc.ts. The generated mbfc.json is also committed and does not require a runtime download.

18. API Rate Limiting

Simple in-memory rate limiting on /api/check prevents accidental Gemini quota burn during the demo, especially once the extension is installed and multiple people in the room start using it simultaneously.

```
const rateLimiter = new Map<string, number>();
```

```
export async function POST(req: Request) {
  const ip = req.headers.get("x-forwarded-for") ?? "unknown";
  const lastCall = rateLimiter.get(ip) ?? 0;
  if (Date.now() - lastCall < 5000) {
    return Response.json({ error: "Rate limited" }, { status: 429 });
  }
  rateLimiter.set(ip, Date.now());
  // proceed with pipeline
}
```

5-second window per IP. Generous enough for normal demo use, sufficient to prevent rapid repeat calls burning Gemini quota. Resets on server restart - acceptable for a demo environment.

19. Sub-Claim Verdict Aggregation Rule

When Stage 1 decomposes a compound claim into multiple sub-claims, the moderator agent must combine their individual verdicts into a single compound verdict. The aggregation rule must be explicit and deterministic. The following algorithm is canonical for TruthCast:

Step 1 — Severity ranking (descending):

FALSE > MOSTLY_FALSE > CONFLICTING > MISLEADING > MOSTLY_TRUE > TRUE > UNVERIFIABLE

Step 2 — Compound verdict assignment:

15. If any sub-claim returns FALSE: compound verdict = FALSE if more than half of sub-claims are FALSE, otherwise MISLEADING.
16. If no FALSE but any MOSTLY_FALSE: compound verdict = MOSTLY_FALSE if majority, otherwise MISLEADING.
17. If no false-family but any CONFLICTING: compound verdict = CONFLICTING.
18. If all sub-claims are TRUE or MOSTLY_TRUE: compound verdict = the most severe among them.
19. If any sub-claim is UNVERIFIABLE and others are verifiable: process verifiable ones only and flag the unverifiable sub-claim in the reasoning chain.
20. If all sub-claims are UNVERIFIABLE: compound verdict = UNVERIFIABLE.

Step 3 — Confidence penalty:

Apply the sub-claim penalty already defined in the confidence formula: multiply base confidence by 0.9^(n_sub_claims - 1). A 4-sub-claim compound verdict cannot exceed 73% confidence regardless of evidence quality.

Step 4 — Reasoning chain format for compound claims:

```
// Moderator reasoning chain template for compound claims
"This claim contains [N] verifiable assertions.
 Sub-claim 1 ([text]): [TRUE/FALSE/etc] — [one sentence reason].
 Sub-claim 2 ([text]): [TRUE/FALSE/etc] — [one sentence reason].
 Compound verdict: [LABEL] because [aggregation rationale]."
```

This rule is passed as a system prompt segment to the moderator agent. The moderator reads it from a shared prompt template in packages/shared/constants.ts so it cannot drift between pipeline versions.

20. SQLite Schema

The off-chain index that powers Stage 0 cache lookups and Stage 5 writes. The schema must be defined before any pipeline code is written because Stage 0, Stage 5, and the rate limiter all depend on it. SQLite file lives at the path specified in SQLITE_PATH env var.

```
-- db/schema.sql  (run once at startup via db/init.ts)

CREATE TABLE IF NOT EXISTS verdicts (
  claim_hash       TEXT PRIMARY KEY,    -- SHA-256 hex of normalized claim
  claim_text       TEXT NOT NULL,       -- Original normalized claim text
  verdict_json     TEXT NOT NULL,       -- Full Verdict JSON (Zod-validated)
  verdict_label    TEXT NOT NULL,       -- Denormalized for fast filtering
  confidence       INTEGER NOT NULL,    -- 0-100
  tx_hash          TEXT,                -- Solana transaction signature (null if write
failed)
  checked_at       INTEGER NOT NULL,    -- Unix timestamp
  ttl_policy       TEXT NOT NULL,       -- SHORT | LONG | STATIC
  pipeline_version TEXT NOT NULL        -- e.g. "2.0"
);

CREATE INDEX IF NOT EXISTS idx_checked_at ON verdicts (checked_at);
CREATE INDEX IF NOT EXISTS idx_verdict_label ON verdicts (verdict_label);
```

db/init.ts — runs at server startup before any request is handled:

```
import Database from "better-sqlite3";
import { readFileSync } from "fs";

export const db = new Database(process.env.SQLITE_PATH!);
db.exec(readFileSync("db/schema.sql", "utf8"));
```

Stage 0 cache lookup function:

```
export function getCachedVerdict(claimHash: string): Verdict | null {
  const row = db.prepare(
    "SELECT verdict_json, checked_at, ttl_policy FROM verdicts WHERE claim_hash = ?"
  ).get(claimHash) as any;
  if (!row) return null;

  // TTL check for time-sensitive claims
  const ageSeconds = Math.floor(Date.now() / 1000) - row.checked_at;
  const ttlMap = { SHORT: 604800, LONG: 7776000, STATIC: Infinity };
  if (ageSeconds > ttlMap[row.ttl_policy as keyof typeof ttlMap]) return null;

  return VerdictSchema.parse(JSON.parse(row.verdict_json));
}
```

Stage 5 write function:

```
export function writeVerdict(verdict: Verdict, txHash: string | null) {
  db.prepare(`
    INSERT OR REPLACE INTO verdicts
    (claim_hash, claim_text, verdict_json, verdict_label, confidence,
     tx_hash, checked_at, ttl_policy, pipeline_version)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
  `).run(
    verdict.claim_hash, verdict.claim_text, JSON.stringify(verdict),
    verdict.verdict, verdict.confidence, txHash,
    verdict.checked_at, verdict.ttl_policy, verdict.pipeline_version
  );
}
```

21. Pipeline Emitter & Agent Orchestration Flow

This section specifies the two missing pieces that connect the architecture: the EventEmitter singleton that bridges agent completion events to the SSE stream, and the concrete orchestration sequence that chains all five agents.

21.1 PipelineEmitter Singleton

pipelineEmitter is a Node.js EventEmitter singleton that lives in packages/pipeline/orchestrator.ts. It is imported by both the agent runner (which emits events) and the Next.js SSE endpoint (which listens). Because Next.js API routes and the pipeline run in the same Node.js process in development, a module-level singleton works correctly. In production, this would be replaced with Redis pub/sub — noted as a post-hackathon upgrade.

```
// packages/pipeline/orchestrator.ts
import { EventEmitter } from "events";
```

```
export const pipelineEmitter = new EventEmitter();
pipelineEmitter.setMaxListeners(50); // support concurrent sessions
```

```
export function emitStage(
  sessionId: string,
  stage: string,
  label: string,
  progress: number,
  verdict?: Verdict
) {
  pipelineEmitter.emit(sessionId, { stage, label, progress, verdict });
```

```
}
```

The SSE endpoint in packages/web/app/api/check/stream/route.ts imports pipelineEmitter from this module and calls pipelineEmitter.on(sessionId, handler). Each agent runner calls emitStage() after completing its work and writing its output file.

21.2 Agent Orchestration Sequence

The orchestrator is the entry point called by POST /api/check. It runs agents sequentially, reading the output of each before starting the next. The [END] signal detection and shared memory file handshake are specified here.

```
// packages/pipeline/orchestrator.ts (continued)
import { randomUUID } from "crypto";
import { readFileSync, writeFileSync } from "fs";
import { OpenClawAgent } from "./openclaw-runner"; // thin wrapper
```

```
export async function runPipeline(
  input: { claim?: string; url?: string },
  sessionId: string
): Promise<Verdict> {
```

```
  const pipelineDir = `/tmp/pipeline-${sessionId}`;
  fs.mkdirSync(pipelineDir, { recursive: true });
```

```
  // Stage 0: Cache lookup
  emitStage(sessionId, "cache_check", "Checking truth ledger...", 5);
  const claimHash = normalizeAndHash(input.claim ?? input.url!);
  const cached = getCachedVerdict(claimHash);
  if (cached) {
    emitStage(sessionId, "complete", "Verdict ready (cached).", 100, cached);
    return cached;
  }
```

```
  // Stage 1: Ingestion
  emitStage(sessionId, "ingestion", "Decomposing claim...", 20);
  writeFileSync(`${pipelineDir}/input.json`, JSON.stringify(input));
  await runWithTimeout(
    () => OpenClawAgent.run("ingestion", pipelineDir),
    10_000, "ingestion"
  );
  const claim = JSON.parse(readFileSync(`${pipelineDir}/claim.json`, "utf8"));
```

```
  // Guard: return UNVERIFIABLE immediately if flagged
```

```
if (claim.type !== "FACTUAL_CLAIM") {
  const verdict = buildUnverifiableVerdict(claim, claimHash);
  writeVerdict(verdict, null);
  emitStage(sessionId, "complete", "Verdict ready.", 100, verdict);
  return verdict;
}
```

```
// Stage 2: Research
emitStage(sessionId, "researcher", "Retrieving evidence...", 45);
await runWithTimeout(
  () => OpenClawAgent.run("researcher", pipelineDir),
  25_000, "researcher"
);
const evidence = JSON.parse(readFileSync(`${pipelineDir}/evidence.json`, "utf8"));
```

```
// Stage 3: Conditional debate
if (evidence.agreement_score < 0.8) {
  emitStage(sessionId, "debate", "Running adversarial debate...", 65);
  for (let round = 0; round < 2; round++) {
    await runWithTimeout(
      () => OpenClawAgent.run("debater-pro", pipelineDir),
      30_000, "debater-pro"
    );
    await runWithTimeout(
      () => OpenClawAgent.run("debater-con", pipelineDir),
      30_000, "debater-con"
    );
    const debate = JSON.parse(readFileSync(`${pipelineDir}/debate.json`, "utf8"));
    if (debate.consensus_reached) break; // early exit if agreed
  }
}
```

```
// Stage 4: Moderator
emitStage(sessionId, "moderator", "Synthesizing verdict...", 80);
await runWithTimeout(
  () => OpenClawAgent.run("moderator", pipelineDir),
  20_000, "moderator"
);
const rawVerdict = JSON.parse(readFileSync(`${pipelineDir}/verdict.json`, "utf8"));
const verdict = VerdictSchema.parse(rawVerdict); // throws to Sentry if malformed
```

```
// Stage 5: Publish
emitStage(sessionId, "publisher", "Writing to Solana ledger...", 95);
let txHash: string | null = null;
try {
```

```
    txHash = await runWithTimeout(
      () => writeSolanaVerdict(verdict),
      15_000, "solana_write"
    );
  } catch (err) {
    Sentry.captureException(err, { extra: { stage: "solana_write" } });
    // Non-fatal: continue without on-chain record
  }
  writeVerdict(verdict, txHash);
  await elevenLabsTTS(verdict.reasoning); // non-blocking
```

```
  emitStage(sessionId, "complete", "Verdict ready.", 100, verdict);
  fs.rmSync(pipelineDir, { recursive: true }); // cleanup
  return verdict;
}
```

21.3 OpenClawAgent Runner Wrapper

OpenClawAgent.run() is a thin Node.js wrapper that starts an OpenClaw agent process for a specific agent name, waits for the [END] signal in its stdout, and resolves. This is what runWithTimeout() wraps.

```
// packages/pipeline/openclaw-runner.ts
import { spawn } from "child_process";
```

```
export const OpenClawAgent = {
  run(agentName: string, pipelineDir: string): Promise<void> {
    return new Promise((resolve, reject) => {
      const proc = spawn("openclaw", ["run", "--agent", agentName], {
        env: { ...process.env, PIPELINE_DIR: pipelineDir },
        stdio: ["ignore", "pipe", "pipe"]
      });
```

```
      proc.stdout.on("data", (chunk: Buffer) => {
        if (chunk.toString().includes("[END]")) {
          proc.kill();
          resolve();
        }
      });
```

```
      proc.stderr.on("data", (chunk: Buffer) => {
        Sentry.captureMessage(chunk.toString(), {
          level: "warning",
          extra: { agent: agentName }
```

```
        });
      });

      proc.on("error", reject);
      // timeout handled externally by runWithTimeout()
    });
  }
};
```

Each agent YAML config receives PIPELINE_DIR as an environment variable and uses it as the base path for all file reads and writes. This isolates concurrent pipeline runs from each other by session.

21.4 Fallback: Plain Node.js Scripts

If OpenClaw fails to install or run correctly, each agent has an equivalent plain Node.js script in packages/pipeline/scripts/. The scripts implement identical logic: read input file, call API, write output file, print [END]. The orchestrator swaps to scripts by changing one flag: const runner = USE_OPENCLAW ? OpenClawAgent : ScriptAgent. Both implement the same interface.

22. Risk Register & Fallbacks

Risk	Likelihood	Fallback
Solana devnet congestion/slow tx	Medium — devnet is sometimes slow	Show tx hash instantly; Explorer confirms async. Demo continues without waiting.
Gemini rate limit (60 req/min free tier)	Low for demo (5 pre-selected claims)	Cache all 5 demo claims' verdicts in SQLite at startup. Stage 0 cache hit returns instantly.
OpenClaw setup fails (Node 22+ required)	Low if environment prepared in advance	Fall back to 5 plain Node.js scripts with identical logic. Same architecture, different orchestration layer.
Pipeline latency (4–6 agents sequential)	Medium — ~15–30s for novel claims	Show per-stage progress bar. Cache pre-loads 5 demo claims for instant demo. Streaming UI makes wait feel active.
Silent agent failure produces bad verdict	Medium — empty evidence.json causes moderator to hallucinate	Sentry captures the null-state exception at Stage 5 with full pipeline context. Frontend shows "pipeline error" rather than a garbage verdict.

Debate agents infinite loop	Low if [END] signal implemented correctly	Hard max 2 rounds enforced at orchestration level regardless of [END] signal.

12. Pre-Selected Demo Claims

Never demo with unknown live input. Pre-run these 5 claims through the pipeline before the hackathon and cache their verdicts in SQLite. Select claims that showcase different verdict labels:

Claim	Expected Verdict	Why This Claim
"The Great Wall of China is visible from space with the naked eye."	**FALSE**	Well-documented, multiple Tier A sources (NASA), high confidence score. Demonstrates clean FALSE verdict path.
"Vaccines cause autism."	**FALSE**	WHO, CDC, multiple peer-reviewed studies. Shows high-confidence verdict with strong Tier A source support. Classic viral misinformation.
"NASA confirmed the existence of water ice on the Moon."	**TRUE**	Shows the system handles TRUE claims honestly. Demonstrates it is not a misinformation detector that always says false.
"Social media companies do more harm than good for democracy."	**UNVERIFIABLE**	Opinion claim. Shows checkworthiness filter working. System returns UNVERIFIABLE immediately — demonstrating intellectual honesty.
"5G towers were used to spread COVID-19 and were destroyed in protests across Europe."	**MISLEADING**	Compound claim: towers destroyed = TRUE; 5G spreads COVID = FALSE. Shows atomic decomposition + MISLEADING taxonomy label. Ideal for triggering debate layer.

23. CI/CD Pipeline

TruthCast uses GitHub Actions for continuous integration. The pipeline runs on every push and pull request. Deployment is automatic on merge to main.

23.1 Hackathon CI (Minimal — Ship Fast)

During the hackathon, CI is limited to a TypeScript compilation check. This catches type errors without running a full test suite, which would take too long to set up under time pressure.

```yaml
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "22" }
      - run: npm install
      - run: npx tsc --noEmit
      - run: node packages/pipeline/db/init.ts --dry-run  # validate schema
```

23.2 Post-Hackathon CI (Full Pipeline)

After the hackathon, the CI pipeline expands to include linting, full type checking across all workspaces, build validation, and automated tests.

```yaml
# .github/workflows/ci.yml (post-hackathon)
name: CI
on: [push, pull_request]
jobs:
  typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "22" }
      - run: npm install
      - run: npx tsc --noEmit                        # all workspaces
      - run: npx eslint packages/**/*.ts --max-warnings 0
      - run: npm run build --workspace=packages/web  # Next.js build
      - run: npx vitest run                          # unit tests
```

```yaml
  deploy:
    needs: typecheck
    if: github.ref == "refs/heads/main"
    runs-on: ubuntu-latest
    steps:
```

```
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "22" }
      - run: npm install -g @railway/cli
      - run: railway up --service truthcast-app
        env: { RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN }} }
```

Branch protection rules to configure on GitHub: require CI to pass before merge; require at least 1 reviewer on pull requests to main; disallow force pushes to main.

24. Deployment Architecture

The deployment strategy has two phases with very different constraints. Understanding why they differ prevents wasted time trying to deploy the hackathon version to production infrastructure.

24.1 Hackathon Deployment (Demo Day)

Keep it as simple as possible. Every minute spent on deployment is a minute not building.

Component	Where it runs	Notes
Web app (Next.js)	Vercel (free tier)	Connect GitHub repo. Auto-deploys on push. URL for judges to use.
Pipeline (OpenClaw)	Local machine	Node.js process running locally. OpenClaw cannot run on serverless. ngrok tunnel if needed.
SQLite database	Local file	Pre-cache 5 demo claims at startup for instant judge demos.
Browser extension	Chrome — load unpacked	chrome://extensions → Developer mode → Load unpacked → select packages/extension/
Solana	Devnet (public)	Free. Verdicts publicly visible on explorer.solana.com/?cluster=devnet.

Important Vercel caveat for hackathon: Vercel serverless functions time out at 10 seconds on the free tier. The pipeline runs locally, not in Vercel functions. The Next.js web app on Vercel only serves the frontend UI and acts as a proxy to the local pipeline via the SSE stream. Point NEXT_PUBLIC_PIPELINE_URL to your local machine or ngrok tunnel.

24.2 Post-Hackathon Production Deployment

The key constraint that determines the production architecture: Vercel serverless functions cannot run long-lived SSE connections or persistent Node.js processes. The entire pipeline must run on a server with no timeout ceiling. Railway is the correct choice because it runs code on long-running servers (not serverless), supports persistent SSE connections natively, supports Next.js with one-click deploy, and is significantly cheaper than Vercel for full-stack workloads.

Component	Platform	Rationale
Everything	**Railway (single service)**	Long-running server, no timeout ceiling, native SSE, one-click Next.js deploy. Simpler than split architecture.
Database	Railway Postgres	One-click add in Railway dashboard. Upgrade path from SQLite: swap better-sqlite3 for pg driver, keep same query interface. Or use Turso (SQLite-compatible, distributed).
Extension	Chrome Web Store	See Section 25 for submission process.
Solana	Mainnet (production only)	Switch SOLANA_RPC_URL env var. Requires real SOL for transaction fees (~0.000005 SOL per memo write).

Railway deployment steps:

21. Create Railway account at railway.app.
22. New Project → Deploy from GitHub repo → select the truthcast repo.
23. Railway auto-detects Next.js and configures the build. Set root directory to packages/web.
24. Add all environment variables from .env.example in the Railway dashboard.
25. Add a Railway Postgres database service. Railway injects DATABASE_URL automatically.
26. Set the start command: node packages/pipeline/orchestrator.ts (or npm run start).
27. Configure custom domain (optional): truthcast.tech → Railway domain settings.

Environment variable management: never commit .env files. Use Railway dashboard for production variables, GitHub Secrets for CI/CD variables, .env.local for local development. The .env.example file is the single source of truth for what variables are required.

25. Browser Extension Deployment (Chrome Web Store)

The browser extension is loaded unpacked during the hackathon. Submitting to the Chrome Web Store is a post-hackathon task. The process takes 1-3 days for initial review.

Preparation checklist:

- Google Developer account: $5 one-time fee at chrome.google.com/webstore/devconsole.
- Update manifest.json: set version, description, and add 5 screenshots (1280x800 or 640x400 px) of the extension in action.
- Privacy policy: required for extensions that make network requests. Host a simple page at truthcast.tech/privacy.
- Extension icons: 16x16, 48x48, 128x128 PNG. Add to manifest.json icons field.
- Update host_permissions in manifest.json: replace http://localhost:3000/* with https://truthcast.tech/* for production.

Submission:

28. Zip the packages/extension/ directory.
29. Upload to Chrome Web Store developer dashboard.
30. Fill in store listing: name, description, screenshots, category (Productivity).
31. Submit for review. Initial review: 1-3 business days.
32. Once approved, users can install from the Chrome Web Store directly.

Version management: bump the version field in manifest.json on every release. Use semantic versioning (1.0.0 for initial release). Chrome auto-updates installed extensions when a new version is approved.

PART 2 — PRESENTATION TALKING POINTS

12. Opening Hook (First 30 Seconds)

Open with one of the following before showing any code or architecture. Choose whichever feels most natural to deliver:

Option A — Economic angle:

"Fake news costs the global economy an estimated $78 billion every year. That figure is from 2019 — before the generative AI explosion. Today it is almost certainly higher. The platforms that spread misinformation employ human fact-checkers who take days to respond. By then, the false claim has already reached millions. TruthCast checks a claim in under 30 seconds and writes the verdict permanently to a public blockchain that no one can edit, delete, or manipulate."

Option B — Scale angle:

"Approximately 62 percent of online content has been classified as false or misleading. 86 percent of the global population has been exposed to misinformation. The World Economic Forum rated misinformation as a top global risk in 2025. The entire global fact-checking infrastructure — Snopes, PolitiFact, Reuters Fact Check, all of them — can check perhaps a few dozen claims per day. On any given day, millions of claims go viral. TruthCast is the first system designed to close that gap."

13. Core Statistics for Presentation

Cite these during your problem statement and impact sections. Each has a primary source noted.

$78B annual global economic cost of fake news (CHEQ / Univ. of Baltimore, cited in DemandSage 2026)	**62%** of online content classified as false or misleading (DemandSage 2026)	**86%** of global citizens have been exposed to misinformation (DISA 2025)	**#1** global risk 2025: misinformation and disinformation (World Economic Forum Global Risks Report 2025)
70% of people struggle to trust online info because	**64%** of people worry that AI-generated content will	**<50%** of prominent election-related misinformation	**40%** of content shared on social media is false (DemandSage

they cannot tell if it is AI-generated (Statista / global survey 2025)	influence elections (Statista 2025)	narratives were fact-checked during 2022 US midterms (arXiv 2024)	2026)

14. Why Existing Solutions Are Insufficient

When a judge asks "how is this different from Snopes or PolitiFact?" — use this framework:

Dimension	Snopes / PolitiFact	Fact Protocol / Factom	TruthCast
Speed	Days to weeks	Days (human editor)	**< 30 seconds**
Automation	Human-only	Human verification layer	**Fully autonomous pipeline**
Tamper-proof record	No (editable website)	Yes (blockchain)	**Yes (Solana, queryable)**
Scales with demand	No (human bottleneck)	Partial	**Yes (AI pipeline)**
Decentralized truth DB	No	Partial (no AI automation)	**Yes (cache-first ledger)**

> *Fact-checks as currently developed and distributed are severely inhibited in election contexts by constraints on coverage, speed, and reach. Fewer than half of all prominent election-related misinformation narratives were fact-checked during the 2022 US midterm elections. — arXiv 2412.13280, December 2024*

15. The Novelty Argument

If a judge asks "why hasn't this been built before?" — the answer is a technology convergence argument:

- OpenClaw, the multi-agent orchestration framework this pipeline uses, was first published in November 2025 — four months ago. The framework did not exist before this.
- Gemini's Google Search grounding, which powers Stage 2 evidence retrieval with automatic citation metadata, reached its current capability level in 2025.
- The adversarial debate architecture for fact-checking (DebateCV) was published at WWW 2026 — one month ago. TruthCast implements the most current research finding in this field.

- Solana's AI agent ecosystem (OpenClaw + Solana workflows) has only existed since February 2026, when the framework's blockchain skills were released.

The combination of: (1) a fully autonomous multi-agent pipeline, (2) adversarial debate for ambiguous claims, (3) research-grade source credibility scoring via MBFC, and (4) a cache-first immutable ledger — has never appeared in any prior academic paper, commercial product, or hackathon submission. The prior art (Fact Protocol, Factom, ANSA blockchain) solves the provenance problem but has no AI automation layer. The prior AI fact-checking research solves the accuracy problem but has no decentralized persistence layer. TruthCast combines both.

16. HackPSU Judging Criteria — Explicit Answers

Prepare one sentence for each criterion. Deliver these directly when judges ask:

Criterion	Your Answer
Implementation	*"The pipeline runs end-to-end: paste a claim, watch 6 agent stages fire, hear the ElevenLabs verdict, and click the Solana Explorer link to verify the on-chain record. Every component is functional."*
Clarity	*"The verdict card shows the label, a 0–100 confidence score with a formula breakdown, the 3-sentence reasoning chain, each source with its MBFC credibility score, and a Solana Explorer link. The pipeline is completely transparent."*
Creativity	*"This is not an API wrapper. We built a 6-stage multi-agent pipeline that includes an adversarial debate layer based on a paper published at WWW 2026, a hybrid source credibility model using the MBFC expert-rated dataset, and a cache-first Solana architecture that turns every fact-check into a permanent, queryable truth record."*
Growth	*"We learned OpenClaw multi-agent orchestration (released November 2025), Gemini grounding with citation metadata, Solana's Memo Program for on-chain data persistence, and the academic literature on adversarial debate for fact verification. None of this was familiar before this weekend."*
Impact	*"Misinformation costs the global economy $78 billion per year, was rated the #1 global risk by the World Economic Forum in 2025, and fewer than half of viral false claims are ever fact-checked. TruthCast addresses the core structural failure: there are not enough human fact-checkers to keep up, and the ones that exist produce verdicts too slowly to matter."*

17. Sponsor Prize Challenge Framing

For each prize, prepare one sentence that explicitly maps your project to the challenge's exact language:

Prize	Framing statement for Devpost + oral pitch
Grand Prize	TruthCast is a technically sophisticated, fully functional system that addresses one of the most critical real-world problems in the digital age — misinformation at scale — using a novel combination of multi-agent AI, adversarial debate, research-grade source credibility scoring, and blockchain provenance.
Base44 Challenge	TruthCast directly addresses Base44's challenge: social media faces serious misinformation and lack of meaningful engagement. We built a tool that lets any user fact-check social media claims in real time, combating misinformation while providing transparent, auditable evidence that rebuilds trust in online information.
IST / OpenClaw	TruthCast uses OpenClaw to build a 6-agent autonomous pipeline with shared memory coordination, a custom ingestion skill with checkworthiness classification, and adversarial debate agents that communicate through the ACP protocol. This is one of the most sophisticated OpenClaw multi-agent deployments in the hackathon space.
MLH Gemini	The researcher agent uses Gemini's google_search grounding tool to retrieve real-time evidence with automatic citation metadata, source URLs, and groundingSupports linkage. Gemini also serves as the fallback credibility scorer for domains not in the MBFC dataset, using its trained knowledge of source reputation.
MLH ElevenLabs	Every verdict is delivered as natural human-sounding audio via ElevenLabs TTS — a design choice that makes fact-checking accessible to users who prefer audio, and demonstrates a genuine use of voice AI as an interface layer rather than a decoration.
MLH Solana	TruthCast uses Solana's Memo Program to write cryptographically timestamped, immutable verdict records on-chain — creating a decentralized, tamper-proof fact database that grows with every check. The cache-first architecture queries this ledger before running the pipeline, giving Solana load-bearing architectural significance rather than treating it as an add-on.
Sentry Credits	TruthCast uses Sentry for production-grade error monitoring across the 6-stage pipeline. The two highest-risk operations — the Solana Memo write and the ElevenLabs TTS call in Stage 5 — are instrumented with Sentry.captureException(), ensuring silent failures are caught before they produce incorrect verdicts. The frontend is instrumented via @sentry/nextjs. This demonstrates production engineering discipline beyond a typical hackathon prototype.
.Tech Domain	Register truthcast.tech at get.tech. List on Devpost. This is a 5-minute task that qualifies for the prize.

18. Post-Hackathon Product Vision (For Q&A)

If a judge asks about future plans, deliver this vision concisely:

Short term (0–6 months):

- Browser extension: fact-check any highlighted text on any webpage.
- API endpoint: sell the verdict feed to newsrooms, social platforms, and enterprise compliance teams.
- The on-chain ledger becomes a public good — any developer can query the TruthCast verdict database.

Medium term (6–18 months):

- Semantic claim matching using embedding models to dramatically improve cache hit rate.
- Multi-language support — the MBFC dataset and Gemini grounding both support non-English sources.
- Integration with ClaimReview schema (used by Google, Bing, Facebook) so verdicts appear directly in search results.

Long term:

- The Solana ledger becomes a decentralized, community-owned truth database — resistant to censorship by any single government or corporation.
- The most defensible moat: every claim fact-checked by any user enriches the database for all future users. Scale creates compounding value that is structurally impossible for centralized fact-checkers to replicate.

19. Five-Minute Video Script (Required for Submission)

Record this before noon Sunday. The video is mandatory for prize eligibility.

Segment	Time	Content
Problem	0:00–0:30	Show a real viral false claim on screen. State the $78B stat and the coverage gap. "Human fact-checkers cannot keep up. We built something that can."
Architecture	0:30–1:00	Show the pipeline diagram. Name the 6 stages in 15 seconds. "6 autonomous AI agents, adversarial debate for ambiguous claims, MBFC source credibility scoring, and every verdict written permanently to Solana."
Live demo	1:00–3:30	Demo 1: "The Great Wall is visible from space" → FALSE (high confidence, Tier A sources). Demo 2: "Vaccines cause autism" → FALSE with WHO citation. Demo 3: "Is this opinion?" → UNVERIFIABLE (shows intellectual honesty). Each time: watch stages fire on progress bar, hear ElevenLabs verdict, click Solana Explorer link.

Future	3:30–4:30	Browser extension, API, semantic deduplication. "Every claim checked by anyone, anywhere, enriches the ledger permanently. The more TruthCast is used, the better it gets — and no one can censor or delete it."
Close	4:30–5:00	Names, Penn State, HackPSU Spring 2026. "TruthCast: the first autonomous AI pipeline that fact-checks claims in real time and writes verdicts to an immutable public ledger that no one can edit."

20. Anticipated Hard Questions & Answers

Question	Answer
"What happens if the AI gets the verdict wrong?"	"Every verdict includes the reasoning chain, cited sources with credibility scores, and a confidence number. Low-confidence verdicts are flagged explicitly. The system is designed to be auditable, not authoritative — it gives users the evidence to judge for themselves. The on-chain record is permanent, so corrections can be issued as new verdicts with higher pipeline versions."
"Isn't this just ChatGPT with extra steps?"	"ChatGPT is a single model call. TruthCast is a 6-stage autonomous pipeline with adversarial agents that argue against each other, a research-grade source credibility system using the MBFC expert-rated dataset, and a blockchain layer. The judging criteria explicitly says 'goes beyond being an API wrapper' — this architecture does."
"Why Solana and not a normal database?"	"A database can be edited, deleted, or taken offline by whoever controls it. The entire history of centralized fact-checkers shows that verdicts get quietly updated or retracted. On Solana, once a verdict is written, it is permanent. No company, no government, and no bad actor can change it. That is structurally impossible with any database."
"How do you handle misinformation about events from this week?"	"Gemini's google_search grounding retrieves real-time information at the moment of the check, not from a static training dataset. Claims about current events are handled by querying live sources. The TTL policy in the schema marks time-sensitive verdicts as SHORT, triggering re-verification after 7 days."
"What about politically contested claims where experts genuinely disagree?"	"This is exactly what the CONFLICTING verdict label and the minority_view field handle. When high-quality sources genuinely disagree, TruthCast does not force a FALSE or TRUE verdict. It surfaces the disagreement, states the strongest arguments on both sides, and returns CONFLICTING with low confidence. That intellectual honesty is a feature, not a limitation."

TruthCast — HackPSU Spring 2026

Architecture document compiled March 27, 2026